Tourism Government — Comprehensive Code Mastery & System Architecture Manual

The Definitive 15-Page Textbook Reference: Frontend Hooks, Service Implementations, JPA Hierarchies, Lombok, & Testing

■ **SENIOR BACKEND ARCHITECT HANDBOOK:**
This master class textbook covers all critical aspects of your microservices ecosystem with **REAL code blocks**. It walks through your React frontend components (`Dashboard.jsx`), dynamic JPA Repository hierarchies, Lombok boilerplates, Jakarta DTO validations, detailed line-by-line walks of `NotificationServiceImpl` and `ReportServiceImpl`, Resilience4j circuit breakers, and finishes with a highly polished 5-Minute Spoken Presenter Script.

Chapter 1: Frontend React Code Walkthrough & State Mechanics

Our frontend is built using **React Vite** for high performance. Let's look at the actual code structure inside Dashboard.jsx and analyze how states and REST APIs are managed:

```
const Dashboard = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchDashboardData = async () => {
      try {
        const token = localStorage.getItem('token');
        const axiosConfig = { headers: { Authorization: `Bearer ${token}` } };
        const BASE_URL = 'http://localhost:8383/tourismgov/v1';

        // Fetch statistics concurrently across microservices
        const [sitesRes, programsRes, eventsRes, usersRes] = await Promise.all([
            axios.get(`${BASE_URL}/sites`, axiosConfig).catch(() => ({ data: [] })),
            axios.get(`${BASE_URL}/programs`, axiosConfig).catch(() => ({ data: [] })),
            axios.get(`${BASE_URL}/events`, axiosConfig).catch(() => ({ data: [] })),
            axios.get(`${BASE_URL}/users`, axiosConfig).catch(() => ({ data: [] }))
        ]);

        setData({
            role: localStorage.getItem('role') || 'TOURIST',
            userName: localStorage.getItem('name') || 'User',
            metrics: { heritageSites: sitesRes.data.length, upcomingEvents: eventsRes.data.length }
        });
      } catch (err) { console.error(err); } finally { setLoading(false); }
    };
    fetchDashboardData();
  }, []);
}
```

■ **Dynamic React Concepts Explained:**
• **useState (Line 2-3):** Declares the reactive state variables. When we call `setData()`, React automatically detects the state update and re-renders the DOM elements.
• **useEffect (Line 5):** Runs the asynchronous aggregation API query once when the Dashboard component mounts on screen. It reads the JWT token from `localStorage` to pass authentication filters.
• **Axios Header Integration (Line 8):** Packages the JWT security token in the format: Bearer [token]. This header is intercepted and validated by our Spring Cloud API Gateway.
• **Framer Motion Transitions:** We wrap dashboard container grids inside <motion.div> to execute gorgeous fade-and-rise animations using simple, hardware-accelerated transitions.

Chapter 2: Decoupled ServiceImpl & Business Logic Walks

Let's trace the actual business logic inside your decoupled backend service implementations. Here are the code architectures and technical comments for your modules:

1. Walkthrough of NotificationServiceImpl.java

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class NotificationServiceImpl implements NotificationService {
    private final NotificationRepository notificationRepository;
    private final UserClient userClient;
    private final EmailService emailService;

    @Override
    @Transactional
    public NotificationResponseDTO create(NotificationRequestDTO request) {
        String recipientEmail = null;
        String recipientName = "User";
        try {
            // Dynamic Feign Client call to USER-SERVICE
            UserDTO user = userClient.getUserById(request.getUserId());
            if (user != null) {
                recipientEmail = user.getEmail();
                recipientName = user.getName();
            }
        } catch (Exception e) {
            log.warn("User-Service offline, using default placeholder details");
        }

        Notification notification = Notification.builder()
                .userId(request.getUserId())
                .subject(request.getSubject())
                .message(request.getMessage())
                .status(NotificationStatus.UNREAD)
                .build();

        Notification saved = notificationRepository.saveAndFlush(notification);
        if (recipientEmail != null) {
            emailService.sendNotificationEmail(recipientEmail, recipientName, request.getSubject(), request.getMessage());
        }
        return toDTO(saved, recipientName);
    }
}
```

■ **Decoupled Notification Logic Points:**

• **Feign Call Protection (Line 15):** Wrapped inside a try-catch block. If the User microservice is offline, our Resilience4j circuit breaker redirects the call to our fallback, and our business code continues using placeholder profiles instead of crashing the system transaction.

• **Save and Flush (Line 29):** Saves the notification and immediately flushes the changes to MySQL database to guarantee database synchronization before sending our background email.

2. Walkthrough of ReportServiceImpl.java

```
@Service
@RequiredArgsConstructor
public class ReportServiceImpl implements ReportService {
    private final ReportRepository reportRepo;
    private final UserClient userClient;
    private final SiteClient siteClient;
    private final NotificationClient notificationClient;

    @Override
    @Transactional
    public ReportSummaryDTO generateReport(ReportRequestDTO request) {
        // 1. Role-based Identity Verification
        UserDTO requester = userClient.getUserById(request.getRequesterId());
        if (requester.getRole() == Role.TOURIST) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Tourists cannot generate reports.");
        }

        // 2. Data Aggregation via OpenFeign Dynamic Querying
        StringBuilder sb = new StringBuilder();
        siteClient.getAllSites().forEach(s ->
            sb.append(String.format("SiteID: %d | Name: %s | Status: %s\n", s.getSiteId(), s.getName(), s.getStatus())));

        // 3. Database Persistence of Government Audit Records
        Report saved = reportRepo.save(Report.builder()
                .scope(request.getScope())
                .metrics(sb.toString())
                .generatedByUserId(requester.getUserId())
                .generatedDate(LocalDateTime.now())
                .build());

        // 4. Observer Notification
        try {
            notificationClient.createNotification(NotificationRequestDTO.builder()
                    .userId(requester.getUserId())
                    .subject("Report Compiles Successful")
                    .message("Report generated successfully.")
                    .build());
        } catch (Exception e) { log.error("Notification fallback offline"); }

        return mapToSummaryDTO(saved, requester.getName());
    }
}
```

■ **Decoupled Reporting Logic Points:**

• **Dynamic Verification (Line 13):** Integrates role-based security boundaries at the business layer. If a Tourist attempts this endpoint, an exception is thrown, sending a `403 Forbidden` response to the client.

• **Aggregator Pattern (Line 19):** Collects, compiles, and formats datasets from multiple independent microservices into a structured document summary.

Chapter 3: Relational ORM Mapping & JPA Repository Hierarchies

To save Java objects into relational databases, we use **Spring Data JPA** and **Hibernate**. Let's walk through the actual Java Entity code, the database annotations, and analyze the JpaRepository inheritance tree:

1. Walkthrough of Notification.java Entity

```
@Entity
@Table(name = "notifications")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Notification {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long notificationId;

    @Column(nullable = false)
    private Long userId;

    private String subject;
    private String message;

    @Enumerated(EnumType.STRING)
    private NotificationStatus status;
}
```

2. Walkthrough of JpaRepository Inheritance Tree

```
@Repository
public interface NotificationRepository extends JpaRepository {
    List findByUserIdOrderByCreatedDateDesc(Long userId);
    long countByUserIdAndStatus(Long userId, NotificationStatus status);
}
```

■ **Repository Hierarchy Explained:**
Spring Data JPA repositories follow a structured hierarchy that eliminates custom SQL code writing:
1. **Repository<T, ID>:** The base interface, used to register repository beans in Spring.
2. **CrudRepository<T, ID>:** Inherits from Repository. Automatically provides generic CRUD methods: `save()`, `findById()`, `findAll()`, `deleteById()`.
3. **PagingAndSortingRepository<T, ID>:** Adds methods to retrieve paginated lists and custom sort rules.
4. **JpaRepository<T, ID> (What we used):** Inherits from PagingAndSorting. Adds JPA-specific flushes, batch deletions (`deleteAllInBatch()`), and returns results directly as Java `List` collections instead of generic Iterables.
• **Custom Queries (Line 3):** Spring automatically parses method names (like `findByUserIdOrderByCreatedDateDesc`) and compiles the corresponding SQL query under the hood at runtime!

Chapter 4: Spring Boot DTO Layers, Validations, & Lombok Boilerplate

To guarantee payload safety and compile-time efficiency, we integrate **Jakarta Bean Validations** directly inside our DTO models, and use **Lombok** to eliminate redundant getters and setters. Let's analyze the code:

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class NotificationRequestDTO {
    @NotNull(message = "Recipient userId is mandatory")
    private Long userId;

    @NotBlank(message = "Subject cannot be empty")
    @Size(min = 3, max = 100, message = "Subject must be between 3 and 100 characters")
    private String subject;

    @NotBlank(message = "Message cannot be empty")
    private String message;
}
```

■ **Lombok & Validation Deep Dive:**

• **@Data:** A Lombok shortcut that automatically generates `@ToString`, `@EqualsAndHashCode`, `@Getter`, `@Setter`, and `@RequiredArgsConstructor` during compilation, removing hundreds of lines of boilerplate code.

• **@Builder:** Implements the Builder design pattern, allowing us to initialize fields using fluent syntax: NotificationRequestDTO.builder().userId(1L).build().

• **Jakarta Bean Validations:**

- @NotNull: Rejects request payloads where the field is null.
- @NotBlank: Rejects empty strings, blank spaces, or null inputs.
- @Size: Enforces length constraints, throwing exception before business execution.

Chapter 5: Spring Boot JUnit & Mockito Unit Testing Masterclass

We wrote highly efficient Mockito tests that execute in milliseconds by simulating dynamic, fake repository dependencies without starting local databases or Eureka discovery servers. Let's analyze the actual code structure:

```
@ExtendWith(MockitoExtension.class)
public class NotificationServiceTest {
    @Mock
    private NotificationRepository notificationRepository;

    @Mock
    private UserClient userClient;

    @InjectMocks
    private NotificationServiceImpl notificationService;

    @Test
    public void testCreateNotification_Success() {
        // 1. Prepare Request DTO payload
        NotificationRequestDTO req = NotificationRequestDTO.builder()
                .userId(1L).subject("Alert").message("Test alert message").build();

        // 2. Mock Mockito responses (Stubbing)
        UserDTO user = new UserDTO(1L, "Omkar", "omkar@tourismgov.com", Role.ADMIN);
        when(userClient.getUserById(1L)).thenReturn(user);

        Notification notification = Notification.builder().notificationId(99L).userId(1L).build();
        when(notificationRepository.saveAndFlush(any(Notification.class))).thenReturn(notification);

        // 3. Execute business method under test
        NotificationResponseDTO res = notificationService.create(req);

        // 4. Assert and verify outcomes
        assertNotNull(res);
        assertEquals(99L, res.getNotificationId());
        verify(notificationRepository, times(1)).saveAndFlush(any(Notification.class));
    }
}
```

■ **JUnit & Mockito Annotations & Mechanics Explained:**

• **@ExtendWith(MockitoExtension.class):** Binds Mockito runtime interceptors to JUnit 5, initializing annotated mocks.

• **@Mock:** Tells Mockito to generate a simulated, lightweight fake implementation of our interfaces (like `UserClient`). It bypasses network calls entirely, returning configured values instantly.

• **@InjectMocks:** Instantiates `NotificationServiceImpl` and automatically injects our annotated fakes (`notificationRepository`, `userClient`) into its constructor, matching Constructor Injection patterns.

• **Stubbing (when().thenReturn()):** Defines what mock methods should return. If `getUserById(1L)` is called, it returns our configured mock user instead of query database.

• **verify() (Line 29):** Assures that critical SQL database saves were triggered exactly once (`times(1)`).

Chapter 6: The Ultimate 5-Minute Spoken Verbal Script

Practice this word-for-word spoken presentation script to deliver a highly professional and technically outstanding project demo:

■ 0:00 - 1:00 | Intro & Stack Overview

"Good morning. I am thrilled to present my core modules in the Tourism Government platform. I was responsible for the complete frontend-to-backend design, implementation, and unit testing of three core systems: the Admin Dashboard, the Notification Service, and the administrative Reporting Service.

On the Frontend, we built our application on React Vite for high-speed hot-reloading. We used Tailwind CSS to craft a beautiful, mobile-responsive glassmorphism card UI, and Framer Motion to animate our statistics cards smoothly. On the Backend, we built independent Spring Boot microservices following a Database-per-Service model. We routed all REST requests through a Spring Cloud API Gateway with JWT authorization and registered our instances dynamically on Eureka."

■ 1:00 - 2:30 | Notification & Fault Tolerance

"Let's deep dive into the Notification module. This service listens for site creation or compliance violations in other services. It writes strictly to its own database schema, notificationdb, to prevent table locking, and sends email alerts using Spring Email.

To handle network faults, we configured Feign-level Circuit Breakers with Resilience4j. Our configuration monitors a sliding window of the last 10 requests. If 5 or more fail, the circuit opens, and we route traffic to local fallback classes. Once it enters the Half-Open state, it runs 5 trial requests, requiring at least 3 successes to safely close the circuit. On the React UI, a global NotificationContext manages background polling, dynamically updating the unread bell count badge in real-time."

■ 2:30 - 4:00 | Reporting Service & Security

"Next is the Reporting module. This is a high-security service. When a user requests a custom report, the service calls our User Feign Client to verify their security role. If a Tourist attempts this, our system throws an HTTP 403 Forbidden exception, blocking the action.

If the requester is an Admin, the Reporting Service acts as an aggregator. It queries 5 separate downstream microservices via Feign, processes the data, saves an audit trace to reportdb, and compiles the document into a base64 download stream for our React UI. To verify this complex logic without database dependencies, we implemented robust Mockito unit tests, mocking out JpaRepositories and Feign clients to validate happy paths and security boundaries in milliseconds!"

■ 4:00 - 5:00 | Conclusion & Q&A; Pitch

"In conclusion, by combining isolated database schemas, robust Feign fallbacks, sliding window circuit breakers, and Mockito testing, we built an architecture that is highly modular, secure, and resilient, guaranteeing 100% operational uptime for our government administrators. Thank you so much, and I welcome any questions!"